



**POLITÉCNICO
ESTELLA**

**ASIGNATURA: PROGRAMACIÓN DE SERVICIOS Y
PROCESOS**

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

PSP07 TAREA

Presentado por: Eugen Moga

Fecha: 27/03/2026

ÍNDICE

Creación de certificado con keytool.....	1
Servidor SSL	4
Cliente SSL	8
Probando el funcionamiento.....	10

Creación de certificado con keytool

Primero de todo empiezo creando un certificado de clave publica para utilizar con SSL . para ello abro la terminal y pongo el comando.

Donde la clave publica la llamo claveSsl utilizo el algoritmo RSA y lo guardo en el keystore AlmacenSSL

El comando keytool solo me funciona si le pongo la ruta absoluta donde está el archivo keytool C:\Program Files\Java\jdk-23\bin

```
C:\Windows\System32\cmd.e  x  +  v
C:\Users\Moga\Desktop\NetBeans\PSP07_ServidorSSL>"C:\Program Files\Java\jdk-23\bin\keytool.exe
" -genkey -alias claveSsl -keyalg RSA -keystore AlmacenSSL
Eugen Moga
```

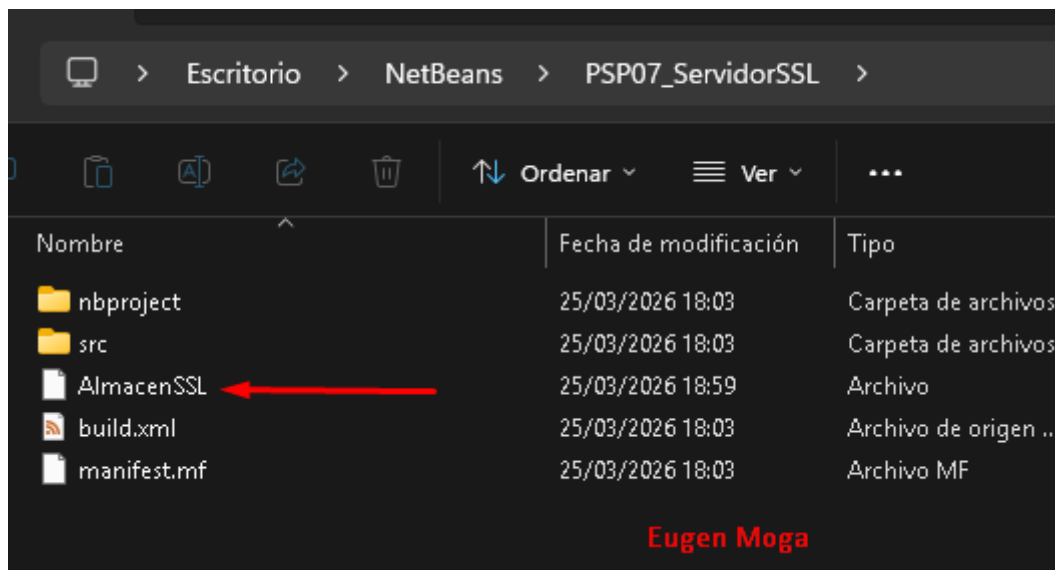
Inserto una contraseña muy segura 123456 y completo todos los datos.

```
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty o
to use the default value in braces.
What is your first and last name?
What is the name of your organizational unit?
What is the name of your organization?
What is the name of your City or Locality?
What is the name of your State or Province?
What is the two-letter country code for this unit?
Is CN=Eugen, OU=Moga, O=Politecnico Estella, L=Cortes, ST=Navarra, C=ES correct?
[no]: yes
Eugen Moga
```

Asi ya tengo generado el certificado qué va a utilizar el servidor SSL

```
[no]: yes
Generating 3.072 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 90
days
for: CN=Eugen, OU=Moga, O=Politecnico Estella, L=Cortes, ST=Navarra, C=ES
C:\Users\Moga\Desktop\NetBeans\PSP07_ServidorSSL>
```

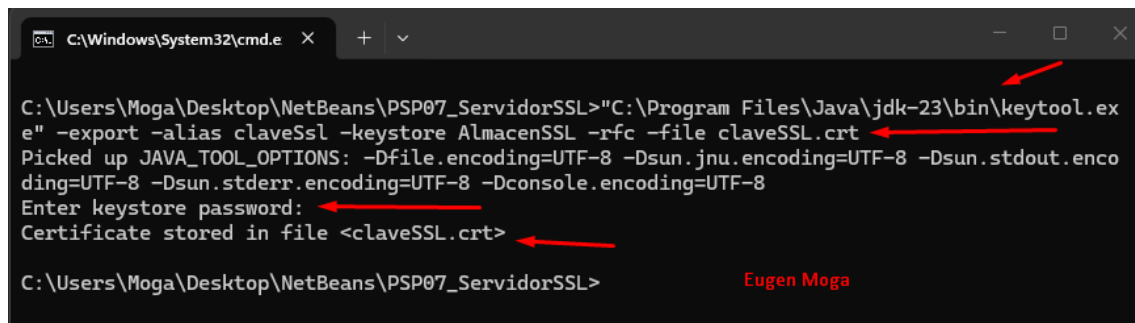
En la carpeta del proyecto ya puedo ver el certificado creado.



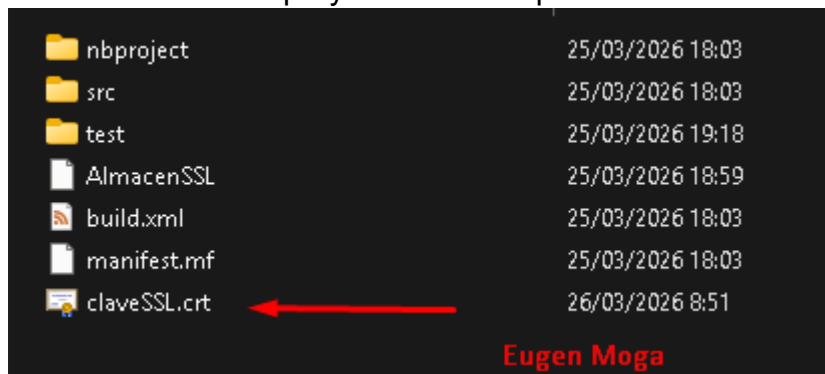
Paso a exportar el certificado claveSsl almacenado en el keystore de nombre AlmacenSSL al fichero claveSSL.crt con el comando

"C:\Program Files\Java\jdk-23\bin\keytool.exe" -export -alias claveSsl -keystore AlmacenSSL -rfc -file claveSSL.crt

Me solicita la contraseña y me genera el certificado



En el directorio del proyecto se ha exportado el certificado.



Después de exportar el certificado a un archivo falta importar el certificado al Truststore para ello importo el certificado claveSSL.crt bajo el nombre claveTSsl al almacen de nombre TrustSSL.

Con el comando: "C:\Program Files\Java\jdk-23\bin\keytool.exe" -import -alias clave_TSsl -file claveSSL.crt -keystore TrustSSL

```
C:\Users\Moga\Desktop\NetBeans\PSP07_ServidorSSL>"C:\Program Files\Java\jdk-23\bin\keytool.exe"
-import -alias clave_TSsl -file claveSSL.crt -keystore TrustSSL
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8
Enter keystore password:
Re-enter new password:
Owner: CN=Eugen, OU=Moga, O=Politecnico Estella, L=Cortes, ST=Navarra, C=ES
Issuer: CN=Eugen, OU=Moga, O=Politecnico Estella, L=Cortes, ST=Navarra, C=ES
Serial number: 4741d68839c301f3
Valid from: Wed Mar 25 18:59:15 CET 2026 until: Tue Jun 23 19:59:15 CEST 2026
Certificate fingerprints:
    SHA1: 5F:A5:3F:BF:4F:83:5B:B2:41:4D:D4:96:C8:67:69:20:43:7A:21:03
    SHA256: 67:EF:B6:E0:25:60:36:05:27:51:36:B5:E2:49:C5:C2:9C:9B:A9:9D:3F:56:8B:C6:F2:8C:
45:9A:A8:40:C6:1F
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

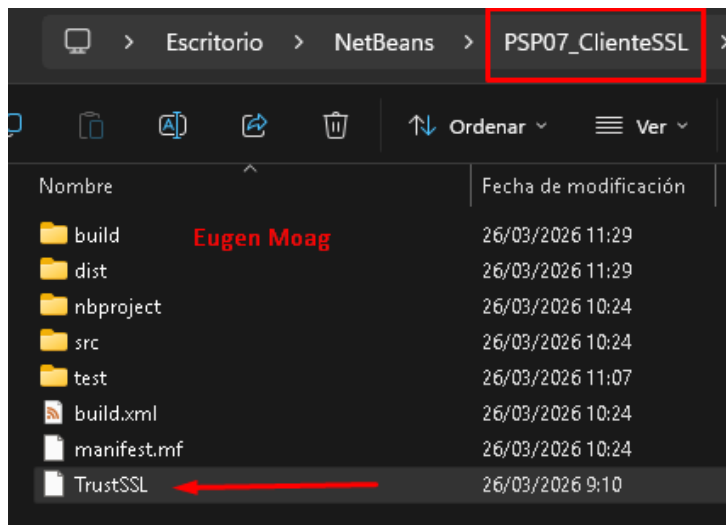
#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 6F 8E 44 F6 FB E8 27 88    1E EC CF CD 07 FE 14 27    o.D...'.....'
0010: 3C CD EE 09                <...
]
]

Trust this certificate? [no]: yes
```

Esto me genera el Truststore en el directorio raíz del proyecto de java

Nombre	Fecha de modificación
nbproject	25/03/2026 18:03
src	25/03/2026 18:03
test	25/03/2026 19:18
AlmacenSSL	25/03/2026 18:59
build.xml	25/03/2026 18:03
claveSSL.crt	26/03/2026 8:51
manifest.mf	25/03/2026 18:03
TrustSSL	26/03/2026 9:10

Este archivo TrustSSL lo voy a mover al directorio del proyecto ClienteSSL porque este archivo es el que utiliza el cliente para validar el certificado del servidor



Servidor SSL

Ahora paso a explicar como he montado el ServidorSSL.

Dentro del método principal creo una estancia vacia de KeyStore de tipo JKS (Java KeyStore formato propietario de Java).

Con FileInputStream creo un flujo de entrada para abrir el archivo AlmacenSSL lo pongo dentro de un bloque try porque si no encuentra el archivo lanza la excepción FileNotFoundException.

ks.load carga el archivo valida la contraseña y descifra el contenido de este modo el KeyStore queda listo para usar.

```
29 public class PSP07_ServidorSSL {
30     public static void main(String[] args) throws Exception {
31
32         // Cargo el keystore del servidor
33         KeyStore ks = KeyStore.getInstance("JKS");
34         try {FileInputStream fis = new FileInputStream("AlmacenSSL");{
35             ks.load(fis, "123456".toCharArray());
36         }
37     }
```

KeyManagerFactory es una clase que administra las claves que se usaran para autenticar el servidor y el cliente.

Creo una instancia de KeyManagerFactory usando el algoritmo SunX509.

Inicializo el KeyManagerFactory con el KeyStore y proporciono la contraseña para acceder a las claves privadas dentro del KeyStore.

```
39 // KeyManagerFactory envuelve el KeyStore para que lo pueda usar el cliente
40 KeyManagerFactory kmf = KeyManagerFactory.getInstance("SunX509");
41 kmf.init(ks, "123456".toCharArray());
42
```

SSLContext es la clase que configura y prepara todo el entorno de seguridad para las comunicaciones TLS/SSL esta clase contiene los componentes necesarios para establecer conexiones seguras.

Creo una instancia de SSLContext que implementa el protocolo TLS (suscesor de SSL obsoleto).

Inicializo el SSLContext con tres componentes, pero yo solo utilizo el primero KeyManagers que es la autenticación del servidor. Los demás los establezco a null.

```
42
43 // SSLContext es el motor TLS. Lo inicio con el protocolo TLS
44 SSLContext ctx = SSLContext.getInstance("TLS");
45 ctx.init(kmf.getKeyManagers(), null, null);
46
```

Creo el servidor de la SSLServerSocket le paso el contexto y le indico el puerto. Para finalizar muestro el mensaje en pantalla que el servidor esta escuchando en el puerto 5000.

```
47 // Creo el socket seguro y espero la conexion del cliente
48 SSLServerSocket servidor =
49     (SSLServerSocket) ctx.getServerSocketFactory().createServerSocket(5000);
50
51 System.out.println(" Servidor escuchando en el puerto 5000 ...");
52
```

Con while true creo un bucle infinito que lo que hace es aceptar conexiones de clientes y llamar al método atender cliente. El servidor se queda a la escucha hasta que lo paro manualmente y puede atender a todos los clientes que le soliciten conexión.

```
53 // Creo el bucle infinito para atender un cliente y luego esperar al siguiente
54 while (true){
55     try (SSLSocket cliente = (SSLSocket) servidor.accept()) {
56         System.out.println(" Cliente conectado.");
57         atenderCliente(cliente);
58     } catch (Exception e){
59         System.out.println(" Error atendiendo el cliente: " + e.getMessage());
60     }
61 }
```

Creo el método atenderCliente y le paso como argumento el socket cliente.

Con KeyGenerator creo un objeto que puede generar claves aleatorias con el algoritmo 3DES (Tripe DES) que es un cifrado simétrico que aplica el algoritmo DES tres veces a cada bloque de datos. (DESede es el nombre Java para 3DES).

Con gen.init inicializo el generador con el tamaño de clave de 168 bits.

Con SecretKey genero la clave y le paso el generador. Para finalizar muestro un mensaje en pantalla que se ha generado la clave.

```
72 public static void atenderCliente(SSLSocket cliente) throws Exception{
73     // Genero la clave
74     // DESede es el nombre Java para 3DES
75     KeyGenerator gen = KeyGenerator.getInstance("DESede");
76     gen.init(168);
77     SecretKey clave = gen.generateKey();
78     System.out.println(" Clave 3DES generada con una longitud " + clave.getEncoded().length);
79 }
```

Con File creo el fichero clave.key y con FileOutputStream creo un flujo de salida para guardar la clave en el archivo. Para finalizar muestro en pantalla el mensaje que se ha guardado la clave.

```
80 // Guardo la clave en un fichero binario
81 // Con getEncoded devuelvo los bytes de la clave en formato binario
82 File fichero = new File("clave.key");
83 try (FileOutputStream fos = new FileOutputStream(fichero)){
84     fos.write(clave.getEncoded());
85 }
86
87 System.out.println(" Clave guardada en el fichero temporal");
88 }
```


Con `DataOutputStream` creo un flujo de salida hacia el cliente y con `WriteLong` le envío la longitud del fichero. Con `salida.flush` vacío el buffer forzando un envío inmediato de datos. Para finalizar muestro el mensaje en pantalla indicando la longitud enviada.

```
89      // Envio la longitud del fichero
90      DataOutputStream salida = new DataOutputStream(cliente.getOutputStream());
91      salida.writeLong(fichero.length());
92      salida.flush();
93      System.out.println(" Longitud enviada: " +fichero.length() + " bytes.");
94
```

Ahora tengo que enviar el fichero al cliente, así que dentro de un bloque `try` que cerrará automáticamente el fichero. Con `FileInputStream` creo un flujo para leer el fichero y le paso como parámetro el fichero.

Después creo el array buffer de tipo `byte` con un tamaño de 1024 posiciones y la variable `leídos` de tipo entero.

Dentro de un bucle `while` lee un bloque del fichero y asigno a `leídos` la cantidad de bytes leídos, al final compruebo si no es -1 (fin de fichero)

Con `salida.write` escribo en el socket los bytes leídos del fichero pasando 3 argumentos (Buffer que contiene los datos, 0 la posición donde empieza, leídos escribe la cantidad de bytes leídos.)

Con `salida.flush` vacío el buffer de salida asegurando que todos los datos se han enviado.

```
95      // Envio el contenido del fichero
96      try {
97          // Leo el fichero por bloques y lo vuelco al socket cifrado por TLS
98          FileInputStream lectorFichero = new FileInputStream(fichero)) {
99              byte[] buffer = new byte[1024];
100             int leídos;
101             while ((leídos = lectorFichero.read(buffer)) != -1){
102                 salida.write(buffer, 0, leídos);
103             }
104         }
105         salida.flush();
106         System.out.println(" Fichero enviado");
107
```

Para finalizar borro el fichero y muestro el mensaje en pantalla

```
109 // Borro el fichero
110 fichero.delete();
111 System.out.println(" Fichero borrado");
112
```

Cliente SSL

Creo otro proyecto y le pongo el nombre PSP07_ClienteSSL que es donde voy a programar en la clase principal dentro del método main.

Aquí voy a definir KeyStore y TrustStore:

TrustStore es un almacén de certificados en los que el cliente confía, mientras que el KeyStore del servidor contiene su propia identidad (certificado más clave privada). El TrustStore del cliente contiene los certificados de las entidades en las que confía.

Primero creo una instancia vacía de KeyStore de tipo JKS (Java KeyStore) aquí tengo que remarcar que este KeyStore en el cliente actuará como TrustStore de hecho creo el flujo de entrada con FileInputStream y le cargo el archivo TrustSSL (después de crearlo lo moví a la raíz del proyecto Cliente) y le paso la contraseña.

```
22 public class PSP07_ClienteSSL {
23     /**
24      * @param args the command line arguments
25      */
26     public static void main(String[] args) throws Exception{
27
28         // Cargo el Truststore del cliente
29         // Fichero TrustSSL
30         // Contiene el certificado del servidor para verificar que es legitimo.
31         KeyStore ks = KeyStore.getInstance("JKS");
32         try (FileInputStream fis = new FileInputStream("TrustSSL")){
33             ks.load(fis, "123456".toCharArray());
34         }
35     }
```

Creo una instancia de TrustManagerFactory usando el algoritmo SunX509 que es un algoritmo de confianza de Sun/Oracle para certificados X509 básicamente lo que hace es que prepara un gestor para validar los certificados recibidos.

Inicializo el TrustManagerFactory con el KeyStore que contiene los certificados confiables TrustStore

```
// TrustManagerFactory envuelve el TrustStore para que lo use El SSLContext
TrustManagerFactory tmf = TrustManagerFactory.getInstance("SunX509");
tmf.init(ks);
```

Creo una instancia SSLContext para el protocolo TLS esto lo que hace en el cliente es preparar el motor SSL para validar servidores.

Igual que el servidor inicializo el SSLContext con tres parámetros, pero en este caso el primer parametro lo pongo en null (KeyManager no necesito autenticarme), segundo parámetro le paso el TrustManagerFactory para validar el servidor, tercer parámetro le establezco a null SecureRandom usa el generador por defecto.

```
SSLContext ctx = SSLContext.getInstance("TLS");
ctx.init(null, tmf.getTrustManagers(), null);
```

Con SSLSocket creo el socket para conectarme al servidor. Donde le paso el contexto y el nombre del servidor junto con el puerto donde escucha.

```
// Conectar al servidor
SSLSocket socket =
    (SSLSocket) ctx.getSocketFactory().createSocket("localhost", 5000);
System.out.println(" Conectado al servidor por TLS.");
```

Con DataInputStream creo un flujo de entrada para recibir los datos del servidor. Creo la variable longitud y le asigno el valor de la entrada readLong que envía el servidor. Muestro en pantalla el mensaje con la longitud de los datos que ha enviado el servidor.

```
// Leo la longitud del fichero
DataInputStream entrada = new DataInputStream(socket.getInputStream());
long longitud = entrada.readLong();
System.out.println(" El servidor envia " + longitud + " bytes.");
```

Creo el array datos donde le indico la longitud que debe leer. Y con entrada readFully leo todos los datos y muestro el mensaje en pantalla que se han recibido los datos correctamente.

```
// Leeo todos los datos
// readFully garantiza que se leen todos los datos esperados
byte[] datos = new byte[(int) longitud];
entrada.readFully(datos);
System.out.println(" Datos recibidos correctamente.");
```

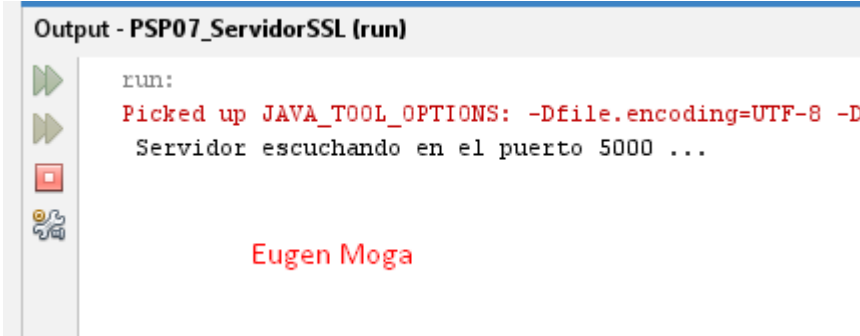
Con FileOutputStream creo un flujo de salida para guardar los datos localmente en el archivo clave_recibida.key. con write escribo los datos en el archivo y luego cierro el flujo de salida con close. Tambien muestro en pantalla el mensaje que la clave se ha guardado y pongo el nombre del archivo. Para finalizar cierro el socket y el programa cliente deja de estar en ejecución.

```
60      // Guardo el fichero en local
61      FileOutputStream fos = new FileOutputStream("clave_recibida.key");
62      fos.write(datos);
63      fos.close();
64      System.out.println(" La clave se ha guardado {clave_recibida.key}");
65
66      socket.close();
67  }
68  }
```

Eugen Moga

Probando el funcionamiento.

Para probar el funcionamiento primero ejecuto el servidor y veo que me muestra el mensaje que el servidor esta escuchando en el puerto 5000.

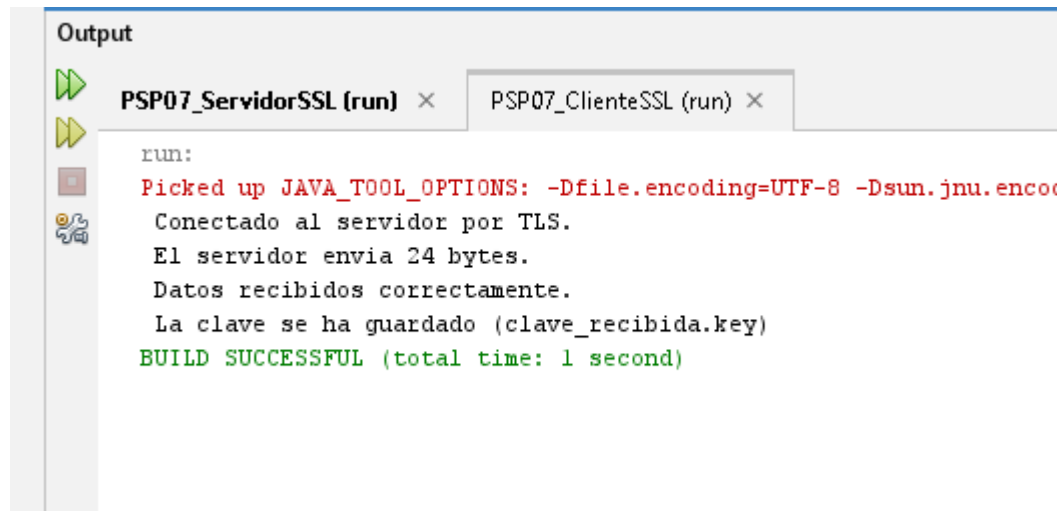


Output - PSP07_ServidorSSL (run)

```
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -D
Servidor escuchando en el puerto 5000 ...
```

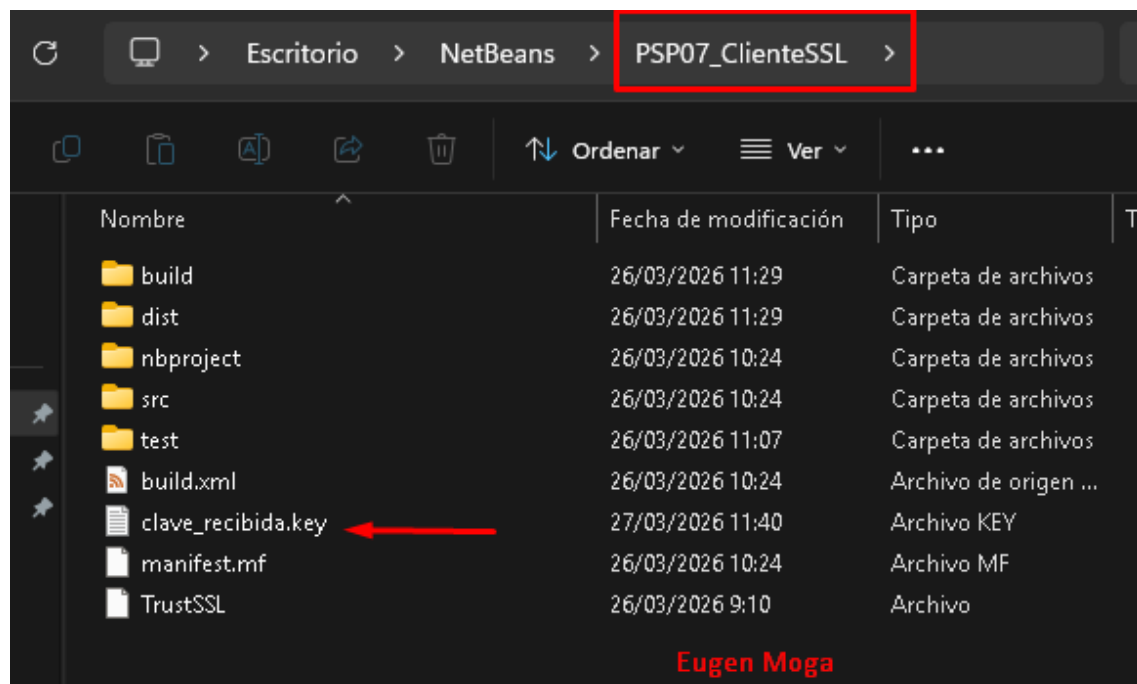
Eugen Moga

Ahora ejecuto el cliente, puedo ver los mensajes que me va mostrando durante la ejecución.

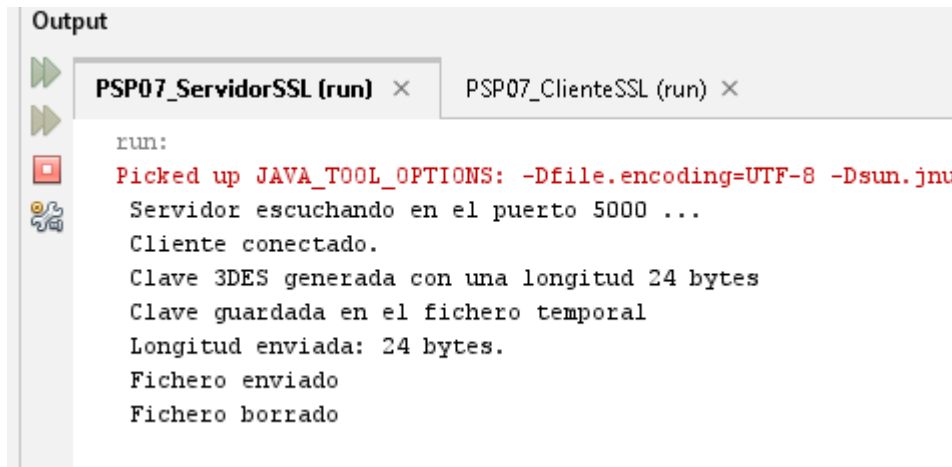


```
Output
PSP07_ServidorSSL (run) × PSP07_ClienteSSL (run) ×
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8
Conectado al servidor por TLS.
El servidor envia 24 bytes.
Datos recibidos correctamente.
La clave se ha guardado (clave_recibida.key)
BUILD SUCCESSFUL (total time: 1 second)
```

Esto me acaba de generar el archivo clave_recibida.key en el directorio raíz del proyecto cliente de netbeans.



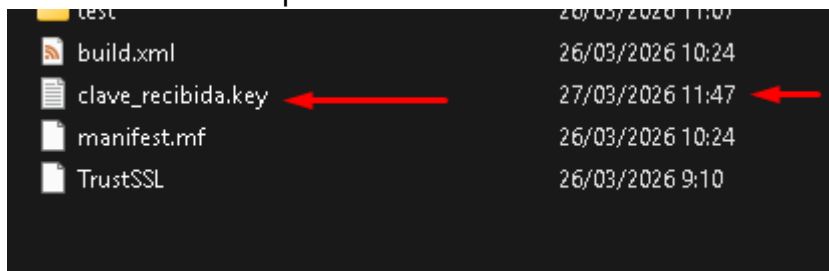
Mientras el cliente se ha conectado al servidor ha sido atendido por el servidor y a finalizado su ejecución. El servidor queda en ejecución esperando al siguiente cliente.



The screenshot shows an IDE's Output window with two tabs: 'PSP07_ServidorSSL (run)' and 'PSP07_ClienteSSL (run)'. The 'PSP07_ServidorSSL (run)' tab is active, displaying the following output:

```
run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu
Servidor escuchando en el puerto 5000 ...
Cliente conectado.
Clave 3DES generada con una longitud 24 bytes
Clave guardada en el fichero temporal
Longitud enviada: 24 bytes.
Fichero enviado
Fichero borrado
```

Ahora si vuelvo a ejecutar al cliente el servidor vuelve atenderle generando la clave 3DES y enviándola al cliente. Se puede ver que ha vuelto a generar el archivo a una hora posterior.



The screenshot shows a file explorer window with a list of files and their creation dates. A red arrow points to the file 'clave_recibida.key', which was created on 27/03/2026 at 11:47. Another red arrow points to the date '27/03/2026 11:47'.

File Name	Creation Date
test	26/03/2026 11:07
build.xml	26/03/2026 10:24
clave_recibida.key	27/03/2026 11:47
manifest.mf	26/03/2026 10:24
TrustSSL	26/03/2026 9:10